

Artificial Intelligence

AI-2002

Lab 07

CSP

National University of Computer & Emerging Sciences – NUCES –
Karachi



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

Course Code: AI-2002	Artificial Intelligence Lab
-----------------------------	------------------------------------

1. Objective	3
2. Introduction to CSP	3
3. Assignments and Domains in CSP	3
3.1 Consistent (Legal) Assignment	4
3.2 Complete Assignment	4
3.3 Partial Assignment	5
4. Domain	
4.1. Discrete Domain	
4.2. Finite Domain	5
4. Code Examples	5



1. Objective

1. Learn how to represent real-world problems as variables, domains, and constraints.
2. Gain knowledge of backtracking search, constraint propagation, and heuristics to solve CSPs efficiently.

2. Introduction to CSP

To satisfy the constraint according to the set of rules is known as constraint satisfaction problem. CSP plays an important part in Artificial Intelligence with csp's all the decision making problems solved easily when its values meet specific constraints or rules of the problem.

The CSP is defined through the three variables **X** (set of variables) , **D** (set of domains) **where** each Variables has specific domains and **C** (set of constraints)

Constraint values can be represented as a **pair{scope, rel}** where:

Scope is a tuple of variables present in a constraint and **Relation** is a combination or set of allowed values of the variables.

3. Assignments and Domains in CSP:

1. Consistent (Legal) Assignment :

An assignment is consistent if it does not violate any constraints of the problem. In other words, all the assigned values satisfy the given constraints.

For example: Map Coloring Problem in the game of LUDO all players can take unique color in consistent (Legal) Assignment suppose four players are playing this game Player_1 = Red , Player_2 = Yellow , Player_3 = Green , Player_4 = Blue so the variables , domains and constraints should be:

Variables: {Player_1, Player_2, Player_3, Player_4}

Domains: {Red, Blue, Green, Yellow}

Constraints: Player_1 $\neq$ Player_2, Player_2 $\neq$ Player_3, Player_1 $\neq$ Player_3

2. Complete Assignment:

A complete assignment should be where each and every variable assigns a value



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

and the solution is to satisfy the constraints.

For example: Sudoku Game in which a complete assignment assigns each row and column a unique value containing values to all variables.

1	4	3	2
2	3	4	1

Variables: Each cell in the grid.

Domains: Possible values {1,2,3,4} (or {1-9} for a 9x9 Sudoku).

Constraints: Uniqueness in rows, columns, and sub-grids.

Goal: Find a **complete and valid assignment** satisfying all constraints.

3. Partial Assignment:

Those assignments who assign values to some variables are known as partial assignments.

For example: In Sudoku Game Partial Assignment assigns values to some variables not all.

—	1	—	3	—
2	—	4	—	3

4. Domains:

Domains are used by the variables



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

Discrete Domain:

This domain is infinite where multiple variables can be mapped to one state For Example: Multiple Variables can use start state infinite times.

Finite Domain:

A finite domain containing one domain for one variable can be a specific variable in a form of continuous states.

Constraints in CSP:

1. Unary Constraints:

It is the simplest type of constraint that restricts the value of a single variable.

2. Binary Constraints:

Binary means “two” variables lies in this constraint where the value of one variable contains the set of variables values that shows the relationship between two variables for example:if x_2 is constrained by x_1 and x_3 , it means that the possible values of x_2 are influenced by or lie within a set defined by x_1 and x_3 .

3. Global Constraints:

The variables are arbitrary in this constraint and have no relationship or connection or pattern between them.

4. Linear Constraints:

The variables in this constraint contain integer values in linear form.

5. Non-linear Constraints:

The variables in this constraint contain integer values in a non-linear form.

6. Constraint propagation in CSP:

It is a special type of inference that helps in reducing the legal number of values for the variables and makes the propagation local consistent in the form of nodes. We solve the constraint satisfaction problem with this inference.

7. Node Consistency:

A variable is considered node consistent if every value in its domain satisfies the unary constraints imposed on it.

8. Arc Consistency:

A variable is arc consistent if each value in its domain satisfies the binary constraints with respect to the related variables.

9. Path Consistency:

When the evaluation of a pair of variables concerning a third variable can be extended to another variable while satisfying all binary constraints, it is similar to arc consistency.

10.k-consistency:

This type of consistency is used to establish stronger forms of propagation.



5. Code Examples

```
pip install ortools

Collecting ortools
  Downloading ortools-9.11.4210-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.0 kB)
Collecting absl-py>=2.0.0 (from ortools)
  Downloading absl_py-2.1.0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.11/dist-packages (from ortools) (1.26.4)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from ortools) (2.2.2)
Collecting protobuf<5.27,>=5.26.1 (from ortools)
  Downloading protobuf-5.26.1-cp37-abi3-manylinux2014_x86_64.whl.metadata (592 bytes)
Requirement already satisfied: immutabledict>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from ortools) (4.2.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.0.0->ortools) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.0.0->ortools) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.0.0->ortools) (2025.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas>=2.0.0->ortools) (1.17.0)
Downloading ortools-9.11.4210-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (28.1 MB)
  28.1/28.1 MB 65.1 MB/s eta 0:00:00
Downloading absl_py-2.1.0-py3-none-any.whl (133 kB)
  133.7/133.7 kB 12.0 MB/s eta 0:00:00
Downloading protobuf-5.26.1-cp37-abi3-manylinux2014_x86_64.whl (302 kB)
  302.8/302.8 kB 24.3 MB/s eta 0:00:00
```

```
#Import the google ortools library to solve the csp problem
from ortools.sat.python import cp_model

# Declare the model and bind it with CpModel which is already present in
ortools library
model = cp_model.CpModel()

#Declaring the set of variables for csp
num_vals = 3
x = model.new_int_var(0, num_vals - 1, "x")
y = model.new_int_var(0, num_vals - 1, "y")
z = model.new_int_var(0, num_vals - 1, "z")

#Declaring Constraints
model.add(x != y)

#output:-
<ortools.sat.python.cp_model.Constraint at 0x7ec6e989bd50>

solver = cp_model.CpSolver()
status = solver.solve(model)
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"x = {solver.value(x)}")
    print(f"y = {solver.value(y)}")
    print(f"z = {solver.value(z)}")
else:
    print("No solution found.")
```



#Output:-

```
x = 0  
y = 1  
z = 0
```

```
"""Simple solve."""  
from ortools.sat.python import cp_model  
  
def simple_sat_program():  
    """Minimal CP-SAT example to showcase calling the solver."""  
    # Creates the model.  
    model = cp_model.CpModel()  
  
    # Creates the variables.  
    num_vals = 3  
    x = model.new_int_var(0, num_vals - 1, "x")  
    y = model.new_int_var(0, num_vals - 1, "y")  
    z = model.new_int_var(0, num_vals - 1, "z")  
  
    # Creates the constraints.  
    model.add(x != y)  
  
    # Creates a solver and solves the model.  
    solver = cp_model.CpSolver()  
    status = solver.solve(model)  
  
    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:  
        print(f"x = {solver.value(x)}")  
        print(f"y = {solver.value(y)}")  
        print(f"z = {solver.value(z)}")  
    else:  
        print("No solution found.")  
  
simple_sat_program()  
  
# Output :-  
x = 1  
y = 0
```



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

z = 0

```
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count
```

```
solver = cp_model.CpSolver()
solution_printer = VarArraySolutionPrinter([x, y, z])
# Enumerate all solutions.
solver.parameters.enumerate_all_solutions = True
# Solve.
status = solver.solve(model, solution_printer)
```

#output:-

```
x=1 y=0 z=0
x=2 y=0 z=0
x=2 y=0 z=1
x=1 y=0 z=1
x=2 y=1 z=1
x=2 y=1 z=0
x=2 y=1 z=2
x=2 y=0 z=2
x=1 y=0 z=2
x=0 y=1 z=2
x=0 y=1 z=1
```




University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

x=0 y=2 z=1
x=0 y=2 z=2
x=1 y=2 z=2
x=1 y=2 z=1
x=1 y=2 z=0
x=0 y=2 z=0
x=0 y=1 z=0

```
from ortools.sat.python import cp_model
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count

def search_for_all_solutions_sample_sat():
    """Showcases calling the solver to search for all solutions."""
    # Creates the model.
    model = cp_model.CpModel()

    # Creates the variables.
    num_vals = 3
    x = model.new_int_var(0, num_vals - 1, "x")
    y = model.new_int_var(0, num_vals - 1, "y")
    z = model.new_int_var(0, num_vals - 1, "z")
```



```
# Create the constraints.
```

```
model.add(x != y)
```

```
# Create a solver and solve.
```

```
solver = cp_model.CpSolver()
```

```
solution_printer = VarArraySolutionPrinter([x, y, z])
```

```
# Enumerate all solutions.
```

```
solver.parameters.enumerate_all_solutions = True
```

```
# Solve.
```

```
status = solver.solve(model, solution_printer)
```

```
print(f"Status = {solver.status_name(status)}")
```

```
print(f"Number of solutions found: {solution_printer.solution_count}")
```

```
search_for_all_solutions_sample_sat()
```

```
#output:-
```

```
x=1 y=0 z=0
```

```
x=2 y=0 z=0
```

```
x=2 y=0 z=1
```

```
x=1 y=0 z=1
```

```
x=2 y=1 z=1
```

```
x=2 y=1 z=0
```

```
x=2 y=1 z=2
```

```
x=2 y=0 z=2
```

```
x=1 y=0 z=2
```

```
x=0 y=1 z=2
```

```
x=0 y=1 z=1
```

```
x=0 y=2 z=1
```

```
x=0 y=2 z=2
```

```
x=1 y=2 z=2
```

```
x=1 y=2 z=1
```

```
x=1 y=2 z=0
```

```
x=0 y=2 z=0
```

```
x=0 y=1 z=0
```

```
Status = OPTIMAL
```

```
Number of solutions found: 18
```



Code Example 2: This code uses Google OR-Tools' CP-SAT solver for solving a constraint optimization problem using Integer Linear Programming (ILP).

```
%pip install ortools

from ortools.sat.python import cp_model

def main() -> None:
    """Minimal CP-SAT example to showcase calling the solver."""
    # Creates the model.
    model = cp_model.CpModel()

    # Creates the variables.
    var_upper_bound = max(50, 45, 37)
    x = model.new_int_var(0, var_upper_bound, "x")
    y = model.new_int_var(0, var_upper_bound, "y")
    z = model.new_int_var(0, var_upper_bound, "z")

    # Creates the constraints.
    model.add(2 * x + 7 * y + 3 * z <= 50)
    model.add(3 * x - 5 * y + 7 * z <= 45)
    model.add(5 * x + 2 * y - 6 * z <= 37)

    # Defining the Objective function
    model.maximize(2 * x + 2 * y + 3 * z)

    # Creates a solver and solves the model.
    solver = cp_model.CpSolver()
    status = solver.solve(model)

    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
        print(f"Maximum of objective function: {solver.objective_value}\n")
        print(f"x = {solver.value(x)}")
        print(f"y = {solver.value(y)}")
        print(f"z = {solver.value(z)}")
    else:
        print("No solution found.")

    # Statistics.
```



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

```
print("\nStatistics")
print(f"  status    : {solver.status_name(status)}")
print(f"  conflicts: {solver.num_conflicts}")
print(f"  branches  : {solver.num_branches}")
print(f"  wall time: {solver.wall_time} s")
```

```
main()
```

Output:-

Maximum of objective function: 35.0

```
x = 7
y = 3
z = 5
```

Statistics

```
status    : OPTIMAL
conflicts: 3
branches  : 14
wall time: 0.016781573 s
```

```
#Import the google ortools library to solve the csp problem
```

```
from ortools.sat.python import cp_model
```

```
# Declare the model and bind it with CpModel which is already present in
ortools library
```

```
model = cp_model.CpModel()
```

```
#Declaring the set of variables for csp
```

```
num_vals = 3
```

```
x = model.new_int_var(0, num_vals - 1, "x")
```

```
y = model.new_int_var(0, num_vals - 1, "y")
```

```
z = model.new_int_var(0, num_vals - 1, "z")
```

```
#Declaring Constraints
```



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

```
model.add(x != y)
```

```
#output:-
```

```
<ortools.sat.python.cp_model.Constraint at 0x7ec6e989bd50>
```

```
solver = cp_model.CpSolver()
status = solver.solve(model)
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"x = {solver.value(x)}")
    print(f"y = {solver.value(y)}")
    print(f"z = {solver.value(z)}")
else:
    print("No solution found.")
```

```
#Output:-
```

```
x = 0
```

```
y = 1
```

```
z = 0
```

```
"""Simple solve."""
from ortools.sat.python import cp_model

def simple_sat_program():
    """Minimal CP-SAT example to showcase calling the solver."""
    # Creates the model.
    model = cp_model.CpModel()

    # Creates the variables.
    num_vals = 3
    x = model.new_int_var(0, num_vals - 1, "x")
    y = model.new_int_var(0, num_vals - 1, "y")
    z = model.new_int_var(0, num_vals - 1, "z")

    # Creates the constraints.
    model.add(x != y)

    # Creates a solver and solves the model.
    solver = cp_model.CpSolver()
    status = solver.solve(model)
```



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

```
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"x = {solver.value(x)}")
    print(f"y = {solver.value(y)}")
    print(f"z = {solver.value(z)}")
else:
    print("No solution found.")
```

simple_sat_program()

Output :-

```
x = 1
y = 0
z = 0
```

```
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count
```

```
solver = cp_model.CpSolver()
solution_printer = VarArraySolutionPrinter([x, y, z])
# Enumerate all solutions.
solver.parameters.enumerate_all_solutions = True
# Solve.
status = solver.solve(model, solution_printer)
```



#output:-

```
x=1 y=0 z=0
x=2 y=0 z=0
x=2 y=0 z=1
x=1 y=0 z=1
x=2 y=1 z=1
x=2 y=1 z=0
x=2 y=1 z=2
x=2 y=0 z=2
x=1 y=0 z=2
x=0 y=1 z=2
x=0 y=1 z=1
x=0 y=2 z=1
x=0 y=2 z=2
x=1 y=2 z=2
x=1 y=2 z=1
x=1 y=2 z=0
x=0 y=2 z=0
x=0 y=1 z=0
```

```
from ortools.sat.python import cp_model
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count
```



```
def search_for_all_solutions_sample_sat():  
    """Showcases calling the solver to search for all solutions."""  
    # Creates the model.  
    model = cp_model.CpModel()  
  
    # Creates the variables.  
    num_vals = 3  
    x = model.new_int_var(0, num_vals - 1, "x")  
    y = model.new_int_var(0, num_vals - 1, "y")  
    z = model.new_int_var(0, num_vals - 1, "z")  
  
    # Create the constraints.  
    model.add(x != y)  
  
    # Create a solver and solve.  
    solver = cp_model.CpSolver()  
    solution_printer = VarArraySolutionPrinter([x, y, z])  
    # Enumerate all solutions.  
    solver.parameters.enumerate_all_solutions = True  
    # Solve.  
    status = solver.solve(model, solution_printer)  
  
    print(f"Status = {solver.status_name(status)}")  
    print(f"Number of solutions found: {solution_printer.solution_count}")
```

```
search_for_all_solutions_sample_sat()
```

#output:-

```
x=1 y=0 z=0  
x=2 y=0 z=0  
x=2 y=0 z=1  
x=1 y=0 z=1  
x=2 y=1 z=1  
x=2 y=1 z=0  
x=2 y=1 z=2  
x=2 y=0 z=2  
x=1 y=0 z=2
```



x=0 y=1 z=2

x=0 y=1 z=1

x=0 y=2 z=1

x=0 y=2 z=2

x=1 y=2 z=2

x=1 y=2 z=1

x=1 y=2 z=0

x=0 y=2 z=0

x=0 y=1 z=0

Status = OPTIMAL

Number of solutions found: 18

Example # 3: The N-queens Problem In chess, a queen can attack horizontally, vertically, and diagonally. The N-queens problem asks:

How can N queens be placed on an NxN chessboard so that no two of them attack each other? Below, you can see one possible solution to the N-queens problem for N = 4. No two queens are on the same row, column, or diagonal.

Import the libraries

```
import sys
```

```
import time
```

```
from ortools.sat.python import cp_model
```

Declare the model

```
Model = cp_model.CpModel()
```

```
# There are `board_size` number of variables, one for a queen in each column  
# of the board. The value of each variable is the row that the queen is in.
```

```
board_size = 8
```

```
queens = [ model.new_int_var(0, board_size - 1, f"x_{i}") for i in range(board_size)]
```

```
# All rows must be different.
```

```
model.add_all_different(queens)
```

```
# No two queens can be on the same diagonal.
```



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

```
model.add_all_different(queens[i] + i for i in range(board_size))
model.add_all_different(queens[i] - i for i in range(board_size))
```

Add constraints to prevent queens from attacking diagonally.

```
diag1 = []
```

```
for i in range(board_size): # Reintroduce the loop to define 'i'
```

```
    q1 = model.NewIntVar(0, 2 * board_size, 'diag1_%i' % i)
```

```
    diag1.append(q1)
```

Need to define j, assuming you want it to be the same as i:

```
    model.Add(q1 == queens[i] + i)
```

```
model.AddAllDifferent(diag1)
```

Create a solution printer

To print all solutions to the N-queens problem, you need to pass a callback, called a **solution printer**, to the CP-SAT solver. The callback prints each new solution as the solver finds it. The following code creates a solution printer.

```
class NQueenSolutionPrinter(cp_model.CpSolverSolutionCallback):
```

```
    """Print intermediate solutions."""
```

```
    def __init__(self, queens: list[cp_model.IntVar]):
```

```
        cp_model.CpSolverSolutionCallback.__init__(self)
```

```
        self.__queens = queens
```

```
        self.__solution_count = 0
```

```
        self.__start_time = time.time()
```

```
    def print_solution(self):
```

```
        solver = cp_model.CpSolver()
```

```
        solution_printer = NQueenSolutionPrinter(queens)
```

```
        solver.parameters.enumerate_all_solutions = True
```

```
        solver.solve(model, solution_printer)
```

Output:

```
Q _ _ _ _ _
```



```
-----Q-
----Q---
-----Q
-Q-----
---Q----
-----Q--
--Q-----
...91 other solutions displayed...
Solutions found: 92
```

TASK: 01

A **warehouse robot** needs to navigate from its **starting position (S)** to a **target location (T)** in a **grid-based environment**. However, the robot faces the following constraints:

- It can only **move diagonally** (i.e., top-left, top-right, bottom-left, bottom-right).
- The movement cost follows **Pythagoras' theorem** due to diagonal motion.
- Some grid cells contain **obstacles**, which the robot must avoid.
- The robot must find the **shortest valid path** to reach the target.

a) **Define the CSP problem** by specifying:

- Variables
- Domains
- Constraints

b) **Write a Python program** using Google OR-Tools to solve the CSP problem and compute the shortest diagonal path.

c) **Given a 5×5 grid**, where the robot starts at (1,1) and the target is at (4,4), solve the problem and output the path.

TASK: 02

A satellite is monitoring a remote island to assess land erosion. The island is represented as a **grid-based map**, where:

- **1** indicates land.
- **0** indicates water.



University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

To track the island's shrinking boundary, an **AI system** must compute the **track of the largest continuous landmass** on the island. where land meets water. Since erosion changes the land boundary over time, this system will help **environmental scientists** predict future land loss.

1. **Define a constraint model** where each cell is a binary variable (1 for land, 0 for water).
2. **Identify boundary edges** by checking adjacent land and water cells.
3. **Solve the CSP model** using Google's OR-Tools to compute the perimeter efficiently.

Task : 03

Implement a CSP to solve the TSP (Traveling Salesman Problem) for a given set of 10 cities.